AVL Trees

A few points:

A binary search tree allows us to discard one of the subtrees while searching. Based on the key comparison at the root, we could discard the left subtree or the right subtree. Would that mean that the number of comparisons in the worst case will be log(n, 2) where n is the number of nodes in the tree?

The answer is NO. It is possible that the tree could become skewed. The tree could degenerate into a list. In such a case, the number of comparisons while searching will be proportional to the number of elements and will not become logarithmic.

How do we make the tree have the same depth – at most with a difference of 1 – no matter in which direction we move from the root?

This leads to a concept called Height Balanced Binary Search Trees. AVL tree is one of them.

AVL trees:

This tree was invented by two Russian Mathematicians : Adelson-Velsky and Landis.

An **AVL tree** is a **self-balancing Binary Search Tree (BST)**.

AVL stands for **Adelson-Velsky and Landis**, the two researchers who introduced the tree.

The key property is:

For every node, the heights of its left and right subtrees differ by at most 1.

Balance Factor

For every node: h: stands for the height

$$\text{Balance Factor} = h(\text{left subtree}) - h(\text{right subtree})$$

For an AVL tree, the balance factor must be:

$$-1,\ 0,\ +1$$

If the balance factor becomes:

+2 → left-heavy → imbalance

-2 → right-heavy → imbalance

we need a **rotation**.

Example

```
    30
   / \
  20  40
```

If the heights of the left and right subtrees are both 1:

$$BF(30) = 1 - 1 = 0$$

So the node is balanced.

If the balance factor becomes 2 or -2, then we rotate the tree such that the balance factor is 1 or 0 or -1.

For example :

```
    30
   /
  20
 /
10
```

At node 30:

$BF(30) = 2$

After rotation:

```
    20
   / \
  10  30
```

Now:

BF(20) = 0

and the tree is balanced.

Four cases of rotation:

There are four possible imbalance configurations:

Case	Configuration	Rotation
LL	Left-Left	Right rotation
RR	Right-Right	Left rotation
LR	Left-Right	Left rotation + Right rotation
RL	Right-Left	Right rotation + Left rotation

A very useful memory rule is:

Same direction → single rotation
Opposite directions → double rotation

---

1. **LL Rotation**

Example

Insert:

30, 20, 10

The resulting BST is:

```
      30
     /
    20
   /
```

10

The path from the unbalanced node is:

30 → 20 → 10

Both moves are **left**.

Therefore, this is an **LL case**.

Balance factor

At node 30:

Left height = 2

Right height = 0

Therefore:

$$BF(30) = 2$$

The tree is left-heavy.

Solution: Right Rotation

Rotate right around 30:

```
      30
     /
    20
   /
  10
```

```
    ↓
```

```
    20
   / \
  10  30
```

The tree is now balanced.

LL rule

LL → Right Rotation

---

2. **RR Rotation**

Now consider inserting:

10, 20, 30

The BST becomes:

```
  10
   \
    20
     \
      30
```

The path is:

10 → 20 → 30

Both moves are **right**.

Therefore, this is an **RR case**.

At node 10:

$$BF(10) = 0 - 2 = -2$$ The node is right-heavy.

Solution: Left Rotation

Rotate left around 10:

```
  10
   \
    20
     \
      30
       ↓
    20
   / \
  10  30
```

RR rule

RR → Left Rotation

---

3. LR Rotation

This is slightly more interesting.

Insert:

30, 10, 20

The BST becomes:

```
    30
   /
  10
    \
     20
```

The path is:

30 → 10 → 20

The first move is **left**, but the second is **right**.

Therefore:

Left → Right

This is an **LR case**.

Step 1: Left rotation

First rotate the subtree rooted at 10 to the left.

```
    30
   /
  10
    \
```

```
      20
```

becomes:

```
      30
     /
    20
   /
  10
```

Now the tree has become an LL configuration.

Step 2: Right rotation

Rotate right around 30:

```
      30
     /
    20
   /
  10

    ↓

     20
    / \
  10   30
```

The tree is balanced.

LR rule

LR → Left Rotation

followed by

Right Rotation

---

4. **RL Rotation**

Now consider:

10, 30, 20

The BST becomes:

```
  10
   \
    30
   /
  20
```

The path is:

10 → 30 → 20

The first move is **right**, followed by **left**.

Therefore, this is an **RL case**.

Step 1: Right rotation

Rotate right around 30:

```
  10
   \
    30
   /
  20
```

becomes:

```
  10
   \
    20
      \
```

30

Now we have an RR configuration.

Step 2: Left rotation

Rotate left around 10:

10

\

20

\

30

↓

20

/ \

10 30

The tree is balanced.

RL rule

RL → Right Rotation

followed by

Left Rotation

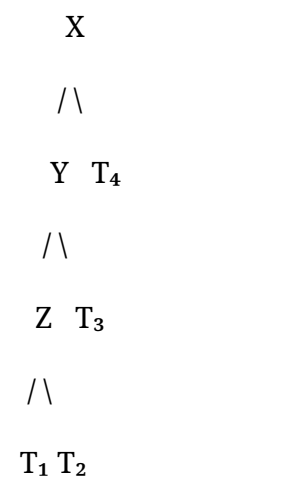
A few points:

- Balancing does not alter the property of the tree. The tree remains to be a BST.
- Rotations changes the shape; but not the inorder listing of the nodes
- Insertion and deletion require log(n, 2) operations.

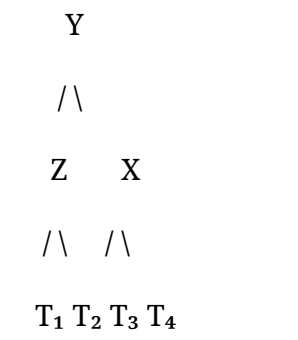
Generalized way of expressing the rotations:

The most useful summary

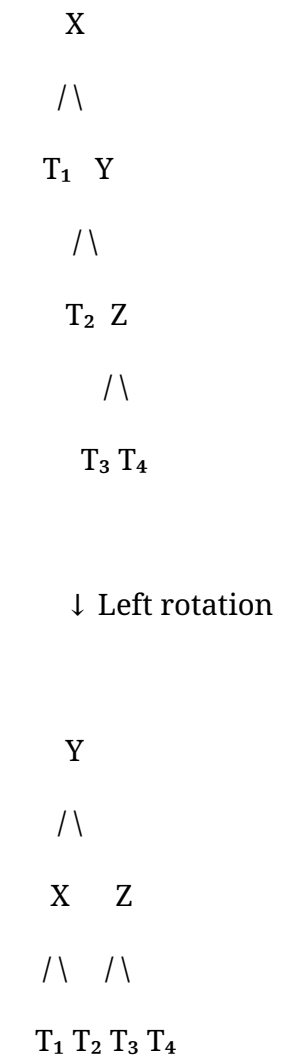
LL



↓ Right rotation

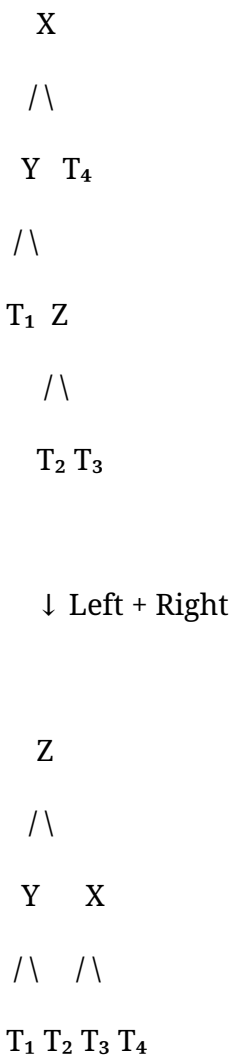


RR



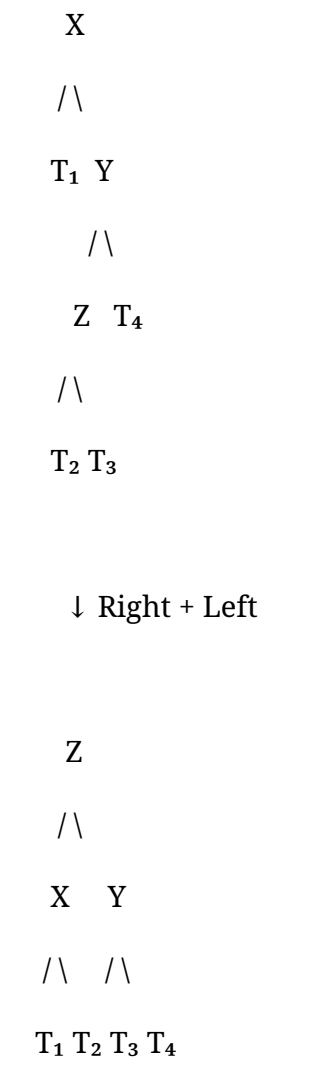
↓ Left rotation

LR



↓ Left + Right

RL



↓ Right + Left

There is another similar tree called the RED BLACK tree which is most commonly used in the libraries.

Amazing idea. Is it not?